

```

/* Custom confirm/alert/prompt dialogs (native confirm()/alert()/prompt()
   are blocked inside sandboxed iframe previews). Depends on t() and esc()
   from app.js. */
/* ===== Confirm / alert / prompt (custom, non-blocking) ===== */
/* Native confirm()/alert()/prompt() are silently blocked in sandboxed
   iframe previews (they just return false/null immediately), which made
   confirmations always cancel themselves. These render a small in-page
   dialog instead, so they work in any environment. */
function ensureModalRoot(){
  let el = document.getElementById('modalRoot');
  if(!el){ el = document.createElement('div'); el.id='modalRoot'; document.body.a
  return el;
}
function showConfirm(message, opts){
  const okLabel = (opts && opts.okLabel) || t('delete');
  const cancelLabel = (opts && opts.cancelLabel) || t('cancel');
  const okClass = (opts && opts.okClass) || 'btn-danger';
  return new Promise(resolve=>{
    const root = ensureModalRoot();
    root.innerHTML = `<div class="modal-overlay"><div class="modal-box">
      <div class="modal-msg">${message}</div>
      <div class="modal-actions">
        <button class="btn btn-ghost" id="modalCancelBtn">${cancelLabel}</button>
        <button class="btn ${okClass}" id="modalOkBtn">${okLabel}</button>
      </div></div></div>`;
    document.getElementById('modalOkBtn').onclick = ()=>{ root.innerHTML=''; reso
    document.getElementById('modalCancelBtn').onclick = ()=>{ root.innerHTML='';
  });
}
function showAlert(message){
  return new Promise(resolve=>{
    const root = ensureModalRoot();
    root.innerHTML = `<div class="modal-overlay"><div class="modal-box">
      <div class="modal-msg">${message}</div>
      <div class="modal-actions">
        <button class="btn btn-primary" id="modalAlertOkBtn">OK</button>
      </div></div></div>`;
    document.getElementById('modalAlertOkBtn').onclick = ()=>{ root.innerHTML='';
  });
}
/* Small text-input dialog. Resolves with the trimmed string the user typed,
   or null if they dismissed/skipped it. Used for the "name this device"
   prompt, but generic enough to reuse anywhere a single line of text is

```

```

    needed. */
function showPrompt(message, opts){
    const okLabel = (opts && opts.okLabel) || t('save');
    const cancelLabel = (opts && opts.cancelLabel) || t('cancel');
    const placeholder = (opts && opts.placeholder) || '';
    const initialValue = (opts && opts.value) || '';
    return new Promise(resolve=>{
        const root = ensureModalRoot();
        root.innerHTML = `

```

/* Forced "refresh now" popup, sent by an admin from the Devices tab.
There is deliberately no cancel / close / tap-outside: the only way
forward is the Refresh button. It lives in its own root (#forceRoot),
above every other popup, so it never wipes a half-filled form -- it just
sits on top of it until the person taps Refresh. Resolves when tapped
(the caller then reloads the page). */

```



```

function ensureForceRoot(){
 let el = document.getElementById('forceRoot');
 if(!el){ el = document.createElement('div'); el.id='forceRoot'; document.body.a
 return el;
}

function showForcedRefresh(title, message, okLabel){
 return new Promise(resolve=>{
 const root = ensureForceRoot();
 root.innerHTML = `
```


```

```

    });
}

/* Popup form used for adding / editing suppliers and items.
opts:
  title      -- heading text (already translated)
  banner     -- optional HTML shown under the title (what's being edited)
  bodyHtml   -- the form fields
  okLabel /  cancelLabel
  againLabel -- optional; adds a "Save & add another" button
  onOpen(box) -- optional; called once the popup is on screen
  onSubmit(again) -- async; runs when Save is tapped. Return:
    {error:'text'}          -> shows the error, popup stays open
    {keepOpen:true, message} -> shows message, clears the fields that
                                have data-clear, popup stays open
    anything else          -> popup closes
The popup only closes via Save or Cancel (tapping outside does nothing),
so a half-typed form is never lost by accident. Resolves when it closes. */
function showFormModal(opts){
  const okLabel = opts.okLabel || t('save');
  const cancelLabel = opts.cancelLabel || t('cancel');
  return new Promise(resolve=>{
    const root = ensureModalRoot();
    root.innerHTML = `
```

```

    setStatus('');
    let res;
    try{ res = await opts.onSubmit(!again); }
    catch(e){ console.error('form submit failed', e); res = {error: t('somethin
    busy = false;
    buttons.forEach(b=>b.disabled = false);
    res = res || {};
    if(res.error){ setStatus(res.error, 'error'); return; }
    if(res.keepOpen){
        box.querySelectorAll('[data-clear]').forEach(el=>{ el.value = ''; });
        const firstField = box.querySelector('[data-clear]');
        if(firstField) firstField.focus();
        setStatus(res.message, 'ok');
        return;
    }
    close();
};
document.getElementById('modalFormOk').onclick = ()=> submit(false);
const againBtn = document.getElementById('modalFormAgain');
if(againBtn) againBtn.onclick = ()=> submit(true);
document.getElementById('modalFormCancel').onclick = close;
box.addEventListener('keydown', e=>{
    if(e.key === 'Enter' && e.target.tagName === 'INPUT'){ e.preventDefault();
    if(e.key === 'Escape') close();
});
if(opts.onOpen) opts.onOpen(box);
const first = box.querySelector('input, select');
if(first) first.focus();
});
}

```